

Rapport

Projet Cloud Computing

Antony Bouyon

Table des matières

Introduction	2
Architecture	2
Infrastructure Azure	2
Organisation des fichiers	2
Réseau et sécurité	2
Base de données managée.....	3
Stockage objet.....	3
Backend API (Node.js / Express)	3
Frontend (React / Vite).....	4
Déploiement	4
Problèmes Rencontrés et Résolutions.....	4
Résultat Final	5

Introduction

Ce projet consiste à concevoir et déployer un backend et le déployer sur Microsoft Azure avec une VM, une Base de données et un stockage Blob.

Architecture

Infrastructure Azure

L'infrastructure est déployée dans la région **France Central** et organisée en 6 fichiers Terraform.

Organisation des fichiers

Fichier	Rôle
main.tf	Resource group, VM, VNet, NSG, IP publique
variables.tf	Déclaration des variables (sensibles marquées)
storage.tf	Storage Account et conteneur blob
database.tf	MySQL Flexible Server avec endpoint privé
identity.tf	Managed Identity et assignation de rôle
outputs.tf	Valeurs exportées post-déploiement
cloud-init.yml	Script de provisionnement de la VM

Réseau et sécurité

Le Virtual Network est segmenté en deux sous-réseaux :

- **subnet-vm**: héberge la VM applicative
- **subnet-mysql**: délégué au service MySQL Flexible Server

Le Network Security Group filtre le trafic entrant sur trois ports essentiels : SSH (22), HTTP (80) et HTTPS (443, pas utilisé).

L'IP publique garantit une adresse fixe pour le déploiement.

Base de données managée

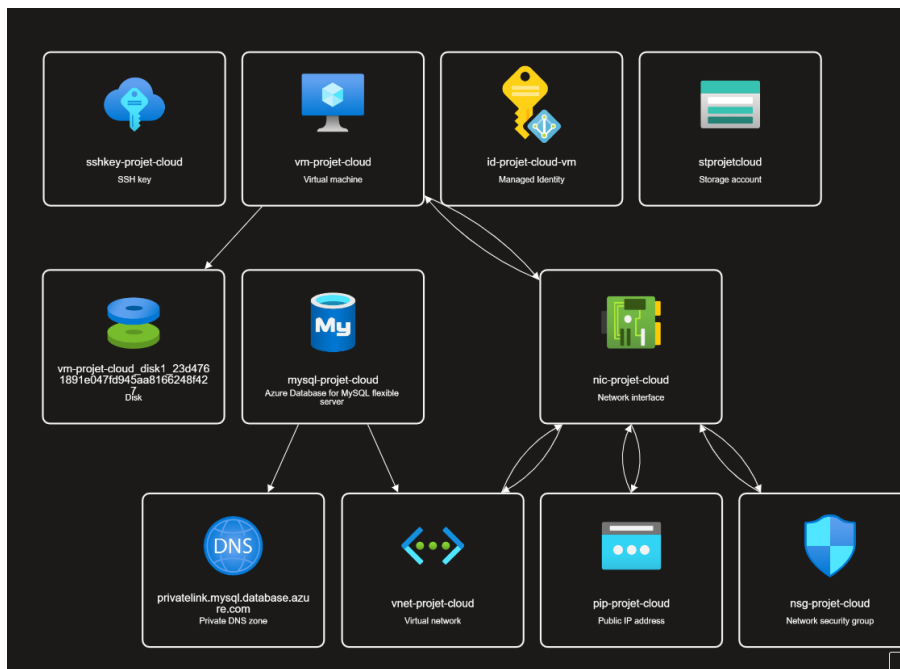
Le serveur MySQL Flexible utilise le tier **B_Standard_B1ms** adapté à un environnement de développement.

L'accès est sécurisé via un **endpoint privé** et une Private DNS Zone, éliminant une exposition publique.

Stockage objet

Le Storage Account héberge le conteneur **user-files** en accès privé.

L'authentification s'effectue via la Managed Identity **id-projet-cloud-vm** dotée du rôle **Storage Blob Data Contributor**.



Backend API (Node.js / Express)

Le backend est une API express simple avec un ORM pour créer et gérer la BDD.

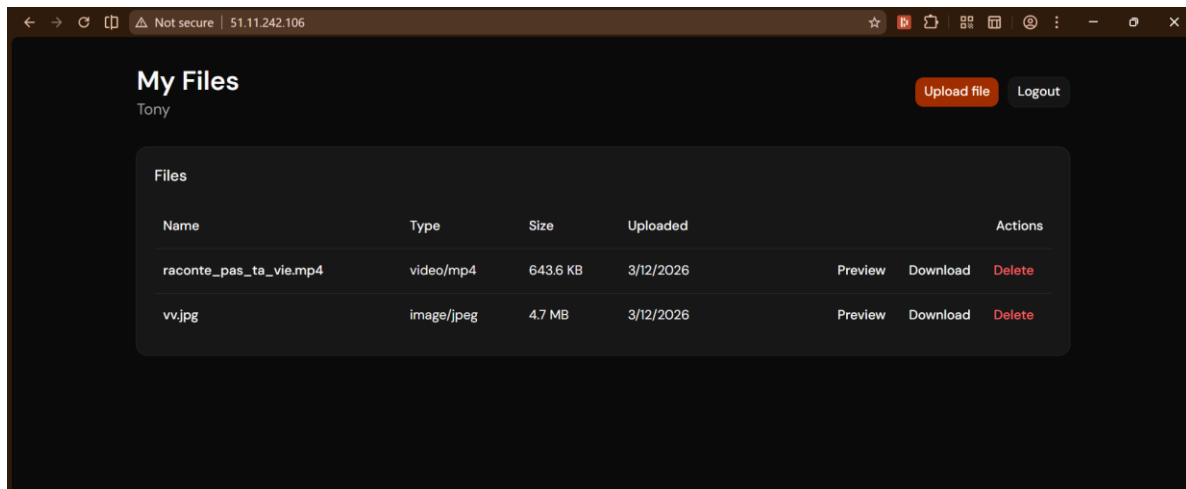
L'authentification combine **Argon2id** pour le hachage des mots de passe et **JWT** signé stocké en cookie **httpOnly**.

Drizzle ORM avec le driver **mysql2** gère les deux tables : users & files

L'upload de fichiers utilise **DefaultAzureCredential** qui détecte automatiquement la Managed Identity en production, tandis qu'en développement il bascule sur les credentials Azure CLI.

Frontend (React / Vite)

Le frontend est une application React simple avec un register/login et un dashboard où on peut upload des fichiers et voir la liste des fichiers que l'on a upload.



Déploiement

Le déploiement reste manuel via script bash pour maîtriser chaque étape :

1. Récupération des outputs Terraform
2. SCP du backend vers /opt/cloud-project/
3. Génération du .env de production via SSH
4. Installation, build et migrations sur la VM
5. Build frontend local + SCP vers /var/www/html/
6. Restart du service systemd nodeapp

Le service systemd **nodeapp** assure le redémarrage automatique en cas de crash.

Problèmes Rencontrés et Résolutions

J'ai pas mal galéré avec les histoires de networking, mais Claude a fini par réussir à tout faire fonctionner correctement.

Aussi j'ai eu un problème avec le SSL mais la solution était simple il suffisait de rajouter `?ssl={}` au connection string...

Résultat Final

L'application déployée offre une **Authentification sécurisée**, une **gestion de fichiers complète** (upload, liste, preview, téléchargement, suppression), et une **architecture cloud** (ressources managées, pas de secret en dur, scaling horizontal possible)

Les améliorations possibles seraient d'ajouter le déploiement continu et de l'observabilité.

Repository : <https://github.com/TonybynMp4/cloud-computing-project>